

Способы оптимизации работы списков с диалогами редактирования

Из этой статьи вы узнаете об основных механизмах Wasaby и методиках разработки, с помощью которых достигается наилучшая скорость открытия диалоговых окон в реестрах. Руководствуйтесь полученными значениями для разработки новых прикладных решений, а также для оптимизации существующих.

В основе данной статьи лежит опыт ускорения открытия карточки товара в разделе "Каталог и цены".

Открытие диалога

В разделе "Каталог и цены" пользователь находит в реестре товаров интересующую запись и путём клика по такой записи вызывает всплывающую панель с карточкой товара. Однако между кликом и до момента отображения панели заметна пауза.

В общем случае, чем больше данных запрашивается перед открытием всплывающей панели, тем больше становится пауза. Поэтому для решения проблемы разработчику необходимо найти компромисс между скоростью доступа к данным и визуальным восприятием плавности открытия панели.

Причины появления паузы и способы их устранения индивидуальны для каждой всплывающей панели. Однако в большинстве случаев справедливы **следующие рекомендации**.

Объединяйте несколько запросов в один

Данные из БД оптимально получать через один запрос в виде иерархической структуры, а затем организовать их дальнейшее распространение до компонентов.

[См. пример объединения запросов к БЛ](#)

Статические данные компонентов лучше запрашивать не по отдельности, а в виде пакетов (bundle), где каждый пакет содержит файлы одной функциональной области. Но и здесь следует соблюсти баланс между количеством пакетов и их размерами. Подробнее о настройке пакетов читайте в разделе [Пакетирование ресурсов](#).

[См. пример пакетирования](#)

Используйте предварительную загрузку статических данных

Когда в реестре предусмотрено открытие всплывающей панели, вы можете предварительно загрузить её статические файлы с помощью RequireJS-плагина **preload!**. Плагин не гарантирует мгновенной загрузки модуля и его зависимостей, однако позволяет предзагрузить файлы, что в результате ускорит открытие всплывающей панели. Подробнее об особенностях работы с плагином читайте в статье [Предварительная загрузка компонентов в зависимостях модуля](#).

Данным способом рекомендуется воспользоваться, когда с большой вероятностью пользователь открывает всплывающую панель при работе с реестром.

[См. пример](#)

Найдите оптимальный способ получения инициализирующих данных панели

Для создания всплывающей панели используйте компонент-контроллер **Controls/form:Controller**. Этот компонент предоставляет разработчику три способа открытия панели, отличающихся алгоритмом получения записи с данными и скоростью отображения такой панели в пользовательском интерфейсе. Выбор способа инициализации данных диалога осуществляется на основе данных, имеющихся на момент [открытия диалога](#), а также от конкретной прикладной задачи.

По умолчанию компонент-контроллер извлекает запись из источника данных самой всплывающей панели, поэтому её открытие откладывается на время подготовки отображаемых данных. Данный способ предполагает построение списка с минимальным количеством данных, но в таком случае панель будет открываться дольше, поскольку всю запись необходимо будет вычитать с БЛ.

Диалог можно открывать мгновенно, если запись с данными подготовлена заранее. Однако при таком способе открытия необходимо проработать формат записи, чтобы он содержал полный набор данных для отображения панели.

Этот вариант сильно замедляет открытие диалога и не рекомендован к использованию.

Следующий способ является самым оптимальным. Для начального отображения диалога используется запись из переданной опции **record**. Такая запись содержит минимальный набор данных, необходимый для построения списка, и несколько полей, которые нужны для первичного открытия карточки. Этих данных должно быть достаточно для первичного отображения диалога. Параллельно открытию диалога происходит вычитывание редактируемой записи с полным набором данных.

Этот способ позволяет содержать в наборе данных для списка минимум полей и достаточно быстро открывать диалог.

Способ инициализации данных диалога настраивается в опции [initializingWay](#). Подробнее о значениях опции и выборе способа инициализации читайте в [статье](#).

Определите компоненты, которые будут показаны первыми

Определите компоненты, первоначальное отображение которых влияет на визуальное восприятие плавности открытия панели. Для таких компонентов данные нужно [вычитывать одним запросом](#) к БЛ приложения.

Для остальных компонентов данные можно запрашивать отложено после построения панели.

Примеры оптимизации карточки товара из раздела "Каталог и цены"

Ускорение открытия карточки с 10 до 4 секунд

Следующие действия существенно ускорили открытие всплывающей панели в разделе "Каталог и цены".

Шаг 1: Создан общий метод БЛ для получения данных к карточке товара

До оптимизации для открытия карточки из интерфейса уходило 12 запросов к источникам данных.

После оптимизации уходит всего один запрос — вызывается метод **Nomenclature.NomenclatureRead**, из которого уже запрашиваются все требуемые данные для открытия карточки. Метод возвращает Record, поля которого уже несут данные для построения карточки товара.

Чтобы посмотреть метод Nomenclature.NomenclatureRead, скачайте файл [Номенклатура.орх](#), откройте его в [Genie](#) и найдите метод.

Шаг 2: Использован RequireJS-плагин preload! для предзагрузки статических ресурсов карточки товара

В разделе "Каталог и цены" для открытия карточки товара применяются методы модуля [Nomenclature/NomenclatureOpen](#). Перед открытием такой карточки клиентом запрашиваются компонент шаблон [NC/Registry/NCNomenclatureCard/NCNomenclatureCard](#) и его зависимости. В данной ситуации их выгодно предварительно загрузить.

Предзагрузка реализована в файле **NomenclatureOpen.js** следующим образом:

На случай, если компонент NC/Registry/NCNomenclatureCard/NCNomenclatureCard не успел загрузиться с помощью preload!, предусмотрена его обычная загрузка в том же файле NomenclatureOpen.js:

Шаг 3: Добавлено пакетирование ресурсов

Выделена функциональная область — карточка товара. Для пакетирования файлов этой области в директории <warehouse/nomenclature>/client/NC/packages создан файл [Cards.package.json](#). В этом файле описано содержимое для функциональных областей:

- **NC/Card/** — компоненты ввода данных (цены, остаток, себестоимость, сумма, закупка и т.д.).

